

Lab 10: Logical Programming Paradigm

1. Objectives

- Understand the theoretical foundation of logic programming.
- Identify and implement Facts, Rules, and Queries.
- Apply logical reasoning to solve real-world problems.
- Write small logic programs in Prolog.

2. Logical Programming Paradigm

Logical programming is a declarative programming paradigm based on formal logic. In this paradigm, a program consists of a set of logical statements that describe facts and rules about a problem domain.

Instead of specifying how to solve a problem (as in imperative programming), logical programming focuses on what relationships hold. The system then uses inference mechanisms to derive answers.

3. Structure of Logical Programs

A Prolog program consists of:

3.1. Facts

Statements that describe relationships that are always true.

```
parent(john, mary).  
parent(mary, anna).  
male(john).  
female(mary).  
female(anna).
```

`parent(john, david).` - This declares that the relationship "parent" exists between the objects "john" and "david". It states that john is the parent of david.

`male(john).` - This defines a property (unary relationship) that "john" is male.

This tells us:

- John is the parent of David
- David is the parent of Sophia
- John and David are males
- Mary, Anna, and Sophia are females

3.2. Rules

Rules are extensions of facts that contain conditional clauses. To satisfy a rule these conditions should be met. They are logical implications that define new relationships.

```
grandparent(X, Y) :-  
    parent(X, Z),  
    parent(Z, Y).
```

Meaning:

X is a grandparent of Y if X is a parent of Z and Z is a parent of Y.

3.3. Queries

Queries are questions you ask the system to see if a specific statement is true or to find information that fits a pattern.

```
?- grandparent(john, anna).
```

When a query is submitted, Prolog employs an inference mechanism to search its database of facts and rules for a match. This process relies on pattern matching, where Prolog attempts to unify the query with existing facts or derive a solution through its rules.

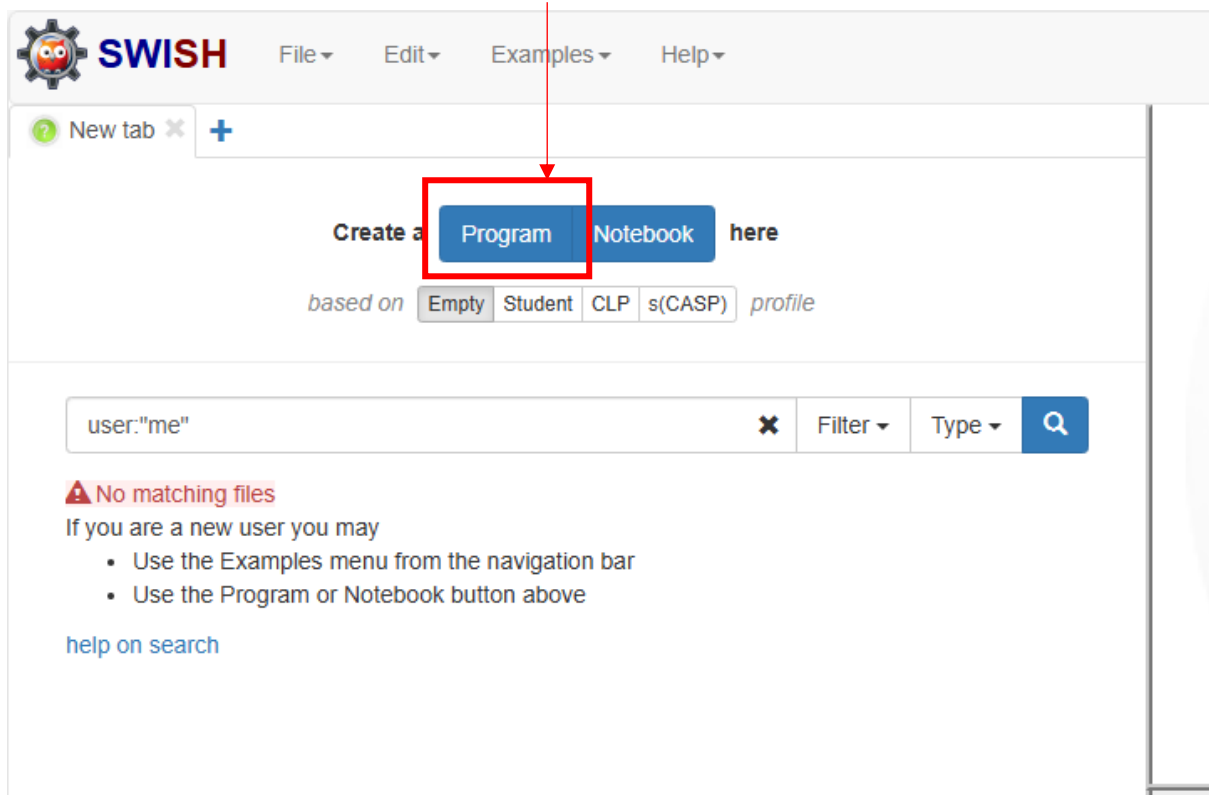
To satisfy this query `grandparent(john, anna) .`, Prolog looks for a person `z` such that `parent(john, z)` and `parent(z, anna)` are both true. John is not a grandparent of Anna, because Anna's parent is not listed in facts. Since no connection exists in the specific fact list provided later in the document, the system returns `false`.

If you ask `parent(john, david) .`, Prolog looks for that exact fact and returns `true`.

If you use a capital letter (a variable), like `parent(john, X) .`, Prolog will find all values that can be "instantiated" (assigned) to `x` to make the statement true and return `X = david`.

Note: You can use the online Prolog environment SWISH (SWI-Prolog) to run your programs (<https://swish.swi-prolog.org/>)

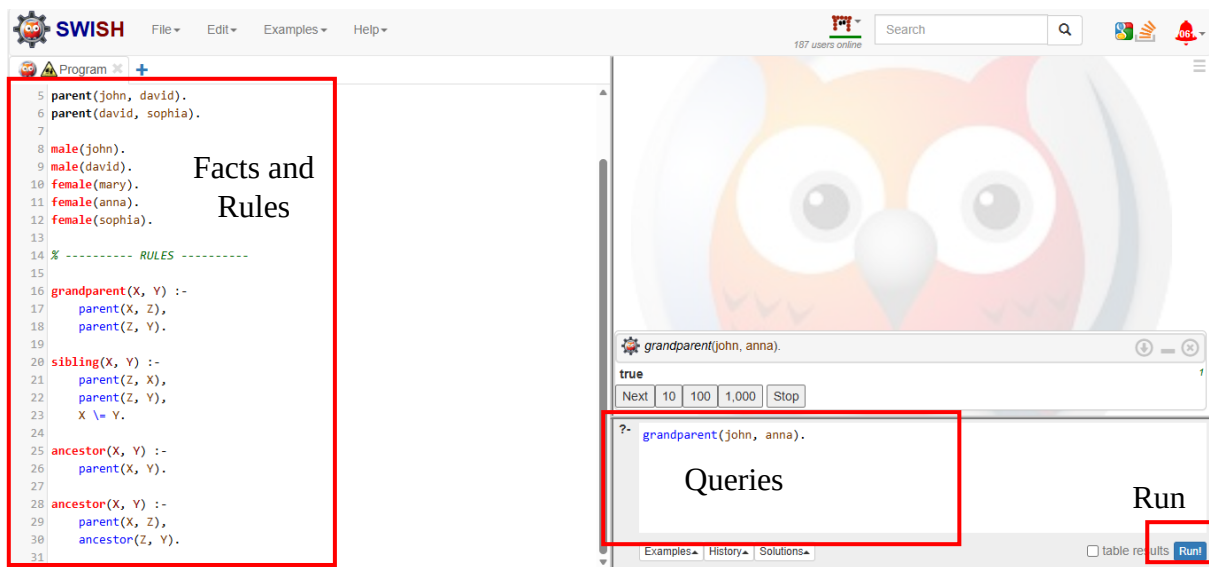
Step 1: Open the link and create a new program.



Step 02: Enter your facts and rules in the left panel (program editor).

Step 03: Enter your Prolog queries in the bottom-right query panel.

Step 04: Click Run to execute the program and view the results.



Facts and Rules

```
parent(john, david).
parent(david, sophia).

male(john).
male(david).
female(mary).
female(anna).
female(sophia).

% ----- RULES -----

grandparent(X, Y) :-
    parent(X, Z),
    parent(Z, Y).

sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.

ancestor(X, Y) :-
    parent(X, Y).

ancestor(X, Y) :-
    parent(X, Z),
    ancestor(Z, Y).
```

Queries

Run the queries one at a time in the query panel.

```
grandparent(john, anna).
```

- To satisfy `grandparent(X, Y)`:
 1. Find `z` such that `parent(john, Z)`. `Z = david`
 2. Check if `parent(david, anna)`. There is no such fact.
- **Result:** `false`
- **Explanation:** John is not a grandparent of Anna, because Anna's parent is not listed in facts.

```
sibling(mary, david).
```

- To satisfy `sibling(X, Y)`:
 1. Find a parent `z` such that `parent(Z, mary)` and `parent(Z, david)`.
 2. We have `parent(john, david)`, but there is **no fact about Mary's parent**.
- **Result:** `false`
- **Explanation:** There's no evidence that Mary and David share a parent, so Prolog returns `false`.

```
ancestor(john, sophia).
```

- `ancestor(X, Y)` works recursively:
 1. Check `parent(john, sophia) → false`
 2. Check `parent(john, Z) → Z = david`, then check `ancestor(david, sophia)`.
 3. Check `parent(david, sophia) → true`.
- **Result:** `true`
- **Explanation:** `John → David → Sophia`. John is an ancestor of Sophia.

Note: In Prolog, any word starting with a **Capital Letter** (like `Sophia`) is treated as a **Variable**, a placeholder that Prolog tries to fill with an answer. A word starting with a **lowercase letter** (like `sophia`) is an **Atom**, a specific, fixed value.

4. Concepts of Logical Programming

Logical programming is built on several core principles:

4.1. Unification

Unification is the process of matching two terms. Unification is a powerful form of pattern matching. Two terms unify if they are identical or if they contain variables that can be instantiated (assigned) in a way that makes the terms identical.

```
% ----- FACTS -----
likes(john, pizza).
likes(john, burger).
likes(mary, pasta).

% ----- RULE -----
friend(X, Y) :-
    likes(X, Z),
    likes(Y, Z),
    X \= Y.
```

Query

```
likes(john, pizza).
```

- **Result:** `true`
- **Explanation:** `john` and `pizza` exactly match the fact.

4.2. Backtracking

Backtracking is how Prolog searches for alternative solutions. When a goal fails, or when you ask for "more," Prolog "backtracks" to the last choice point and tries a different rule or fact.

```
% ----- FACTS -----  
  
color(apple, red).  
color(banana, yellow).  
color(grape, purple).  
color(cherry, red).
```

```
color(Fruit, red).
```

Output:

```
Fruit = apple  
Fruit = cherry
```

Here,

1. Prolog finds `apple`
2. When you press `;` it backtracks
3. Finds `cherry`

That is backtracking.

5. Lab Tasks

5.1. Building a Family Knowledge Base

Extend the provided family tree logic to include more complex relationships.

1. Add at least **four new parent/2 facts** and the corresponding `male/1` and `female/1` facts to extend the given family tree in the example to **three generations**.
2. Define a rule `father(X, Y)` where `X` is the father of `Y`.
3. Define a rule `mother(X, Y)` where `X` is the mother of `Y`.
4. Define a rule `brother(X, Y)` where `X` and `Y` share a parent and `X \= Y`.
5. Define a rule `uncle(X, Y)` where `X` is the brother of `Y`'s parent.

Write your own queries to explore all relationships including `father`, `mother`, `brother`, and `uncle`. For each query, explain the output in your own words.

5.2. Unification and Constraints

In Prolog, the `\=` operator is used to ensure two terms do not unify.

1. Create facts in the form `enrolled(student_name, course_code)` for at least **5 students and multiple courses**.
2. Define a rule `classmates(Student1, Student2)` that identifies students enrolled in the same course.
3. Ensure a student cannot be their own classmate using the `\=` operator.
4. **Query:** Find all classmates of a specific student using a query. Explain the outputs.

5.3. Implementing Recursive Logic

Implement a recursive rule `path(A, B)` for a directed graph defined by facts.

- **Base Case:** There is a path from `A` to `B` if there is a direct edge from `A` to `B`.
- **Recursive Case:** There is a path from `A` to `B` if there is an edge from `A` to some node `Z`, and a path exists from `Z` to `B`.

Test your implementation by writing queries to check whether a path exists between different nodes. Explain the outputs.

6. Submission

Create a PDF file containing screenshots of all your codes and Results along with the explanations. Rename the PDF file as **CO523_Lab10_EXXYYY.pdf**, where EXXYYY represents your E-number.

Upload the file to the FEeLS by the given deadline.